Ch 7.5-7.7: Smoothing Splines, Local Regression, GAMs

Lecture 20 - CMSE 381

Prof. Elizabeth Munch

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Wed, Mar 23, 2022

Announcements

Last time:

- 7.1 Polynomial regression
- 7.2 Step functions
- 7.3 Basis functions
- 7.4 Regression Splines

This lecture:

- 7.5 Smoothing Splines
- 7.6 Local Regression
- 7.7 GAMs

Announcements:

- Mon 3/20: Ch 7.1-7.4
- Weds 3/22: Ch 7.5-7.7
- Friday lab
- Friday Homework #7 due

Section 1

Last time

Basis Functions Setup

$p = 1$

Using X to predict Y

$[f(x)]_+$

Pick a collection of functions b_i and learn the model:

$$y_i = \beta_0 + \beta_1 \underbrace{b_1(x_i)} + \beta_2 \underbrace{b_2(x_i)} + \cdots + \beta_K \underbrace{b_K(x_i)} + \varepsilon_i$$

basis function

Polynomial Regression:

- $b_1(x) = x$
- $b_2(x) = x^2$
- $b_3(x) = x^3$
- $\vdots$
- $b_d(x) = x^d$

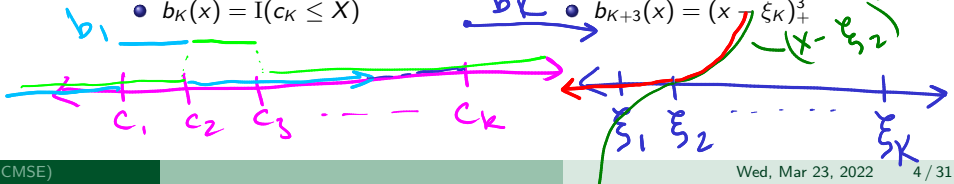
Step functions

- $b_1(x) = \mathrm{I}(X < c_1)$
- $b_2(x) = \mathrm{I}(c_1 \leq X < c_2)$
- $b_2(x) = \mathrm{I}(c_2 \leq X < c_3)$
- $\vdots$
- $b_{K-1}(x) = \mathrm{I}(c_{K-1} \leq X < c_K)$
- $b_K(x) = \mathrm{I}(c_K \leq X)$

Cubic splines

- $b_1(x) = x$
- $b_2(x) = x^2$
- $b_3(x) = x^3$
- $b_4(x) = (x - \xi_1)_+^3 = \begin{cases} (x - \xi_1)^3 & \text{if this is +} \\ 0 & - \end{cases}$
- $\vdots$
- $b_{K+3}(x) = (x - \xi_K)_+^3$

$(x - \xi_2)^3$



Cubic Spline

Knots

each interval

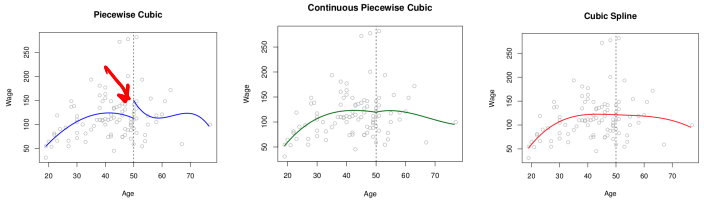
Collection of piecewise cubics

$$f(x) = \begin{cases} \beta_{0,0} + \beta_{0,1}x + \beta_{0,2}x^2 + \beta_{0,3}x^3 & \text{if } x \leq \xi_1 \\ \beta_{1,0} + \beta_{1,1}x + \beta_{1,2}x^2 + \beta_{1,3}x^3 & \text{if } \xi_1 \leq x < \xi_2 \\ \beta_{2,0} + \beta_{2,1}x + \beta_{2,2}x^2 + \beta_{2,3}x^3 & \text{if } \xi_2 \leq x < \xi_3 \\ \beta_{3,0} + \beta_{3,1}x + \beta_{3,2}x^2 + \beta_{3,3}x^3 & \text{if } \xi_3 \leq x < \xi_4 \\ \vdots & \\ \beta_{K,0} + \beta_{K,1}x + \beta_{K,2}x^2 + \beta_{K,3}x^3 & \text{if } x \geq \xi_K \end{cases}$$

Require:

- Continuous
- First derivative continuous
- Second derivative continuous

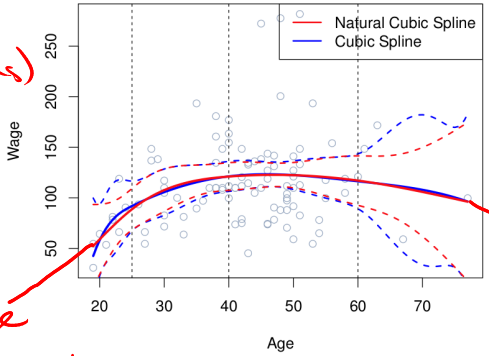
Smooth



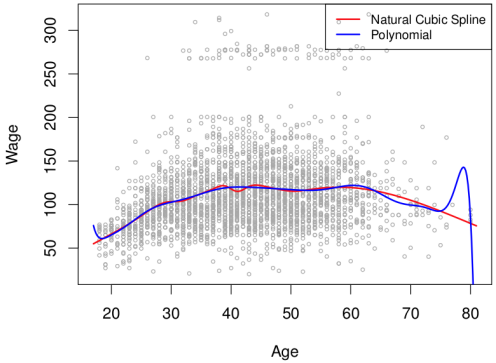
Notes on cubic splines

- Use basis functions (including $(x-\xi_i)_+^3$ fns) to learn cubic splines

- Natural spline
  Outside of the range of the X's, we want a linear function



Cubic splines vs Polynomial Regression



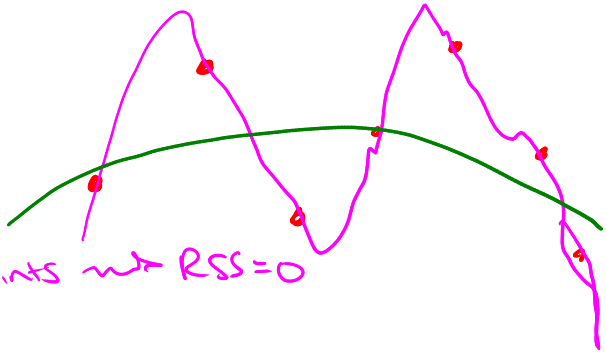
Section 2

Smoothing Splines

The goal

- Say we're looking for a function $g(x)$ that fits observed data well (i.e. $RSS = \sum y_i - g(x_i))^2$ is small).
- With no constraints on g....

Pick a function that interpolates all points $\rightsquigarrow$ RSS=0



- So what we really want is....

Less wiggles

↳ a function both smooth and has small RSS

Smoothing spline

called

Find $\boxed{g}$ that ~~mimizes~~ minimizes

$$\underbrace{\sum_{i=1}^{n}(y_i - g(x_i))^2}_{\boxed{\text{Loss}}: \text{RSS}} + \underbrace{\lambda \int g''(t)^2\, dt}_{\boxed{\text{penalty term}}}$$

tuning parameter

2nd derivative

$|g''(x)|$ small → big

not wiggly

wiggly

Roughness

penalizes global variability in g''

More on the smoothing spline

Find g that minimizes

$$\sum_{i=1}^{n}(y_i - g(x_i))^2 + \lambda \int g''(t)^2\,dt$$

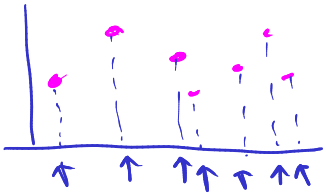
$\lambda = 0$ least squares (penalty has no effect)

$\lambda \to \infty$ g is as smooth as possible $\Rightarrow$ straight line

It turns out: the fcn g that minimizes is

- piecewise cubic
- w/ knots at every x_i in the data set
- continuous
- 1st deriv cont.
- 2nd deriv cont

natural cubic spline



Effective degrees of freedom

- Smoothing spline is a [natural] cubic spline w/ knot at every x_i

  looks like n degrees of freedom } df

  Is this too much flexibility?

- λ controls roughness (opposite of smooth)

  effective degrees of freedom df_λ

- $\lambda \quad 0 \longrightarrow \infty$

  $df_\lambda \quad n \longrightarrow 2$

Computing df_λ

- Fix λ
- Find g that minimizes

smoothing spline

$$\sum_{i=1}^{n}(y_i - g(x_i))^2 + \lambda \int g''(t)^2\, dt$$

- Write $\hat{\boldsymbol{g}}_\lambda$ for the n-vector with fitted values of smoothing spline at training points $x_1, \cdots, x_n$

$\hat{y}_1, \ldots, \hat{y}_n$

computer has equation

- Find $n \times n$ matrix $\boldsymbol{S}_\lambda$ where

$$\hat{\boldsymbol{g}}_\lambda = \boldsymbol{S}_\lambda \boldsymbol{y}$$

vector of outputs $(y_1, \ldots, y_n)$

- The effective degrees of freedom is defined to be

$$df_\lambda = \sum_{i=1}^{n} \{\boldsymbol{S}_\lambda\}_{ii} = \mathrm{trace}(\boldsymbol{S}_\lambda)$$

not nesc. integers

Choosing λ

When in doubt, Cross-Validate

fix λ

get LOOCV

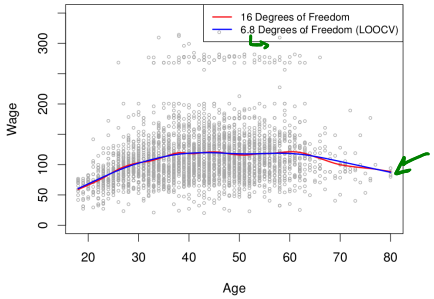
- We don't have to pick knots, only parameter is λ
- Want LOOCV each λ
- Cheap trick to get all λ by fitting once!
- $\hat{g}_\lambda^{(-i)}(x_i)$ $\quad \hat{g}_\lambda^{(-i)}$ learned function on data set w/o the x_i data point

  fitted value for that function
- $\hat{g}_\lambda$ is fitted function all data

$y_i - \hat{y}_i$ $\qquad$ plug in x_i

$$\mathrm{RSS}_{cv}(\lambda) = \sum_{i=1}^{n} (y_i - \hat{g}_\lambda^{(-i)}(x_i))^2 = \sum_{i=1}^{n} \left[\frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{\mathbf{S}_\lambda\}_{ii}} \right]^2$$

An example



Little discernable diff b/t splines

↳ pick the one w/ fewer eff. deg. freedom.

- Red: smoothing spline with 16 effective degrees of freedom.
- Blue: smoothing spline obtained when λ is chosen using LOOCV; result is $df_\lambda = 6.8$ effective degrees of freedom

Section 3

Local Regression

Local Regression Algorithm

k # pts

Algorithm 7.1 *Local Regression At $X = x_0$*

1. Gather the fraction $s = k/n$ of training points whose x_i are closest to x_0.

   percentage

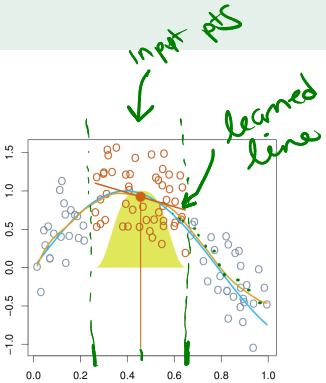
2. Assign a weight $K_{i0} = K(x_i, x_0)$ to each point in this neighborhood, so that the point furthest from x_0 has weight zero, and the closest has the highest weight. All but these k nearest neighbors get weight zero.

3. Fit a *weighted least squares regression* of the y_i on the x_i using the aforementioned weights, by finding $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize

   $y_i - \hat{y}_i$ for linear regression

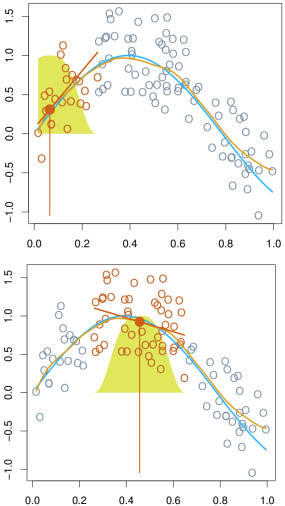
$$\sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2. \tag{7.14}$$

4. The fitted value at x_0 is given by $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.



- Blue line: Generated the data
- Orange: Local regression estimate

More on local regression



- Redo your regression for each x_0 value where you want prediction $\hat{y}_0$
- "Memory based procedure" need all training data every time I want a prediction (k-NN)

Choices to be made for local regression

- Weighting function K
- Choice of local function
  - linear
  - quadratic, cubic
- Span S percentage

  tiny S: local fit, wiggly

  giant S: global fit

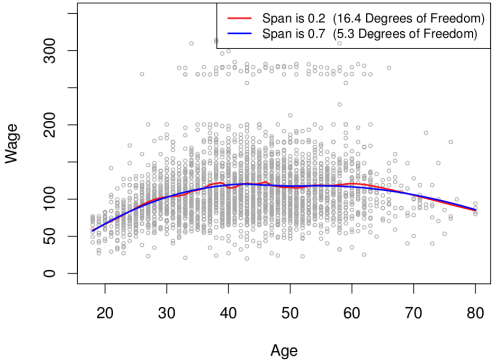
Algorithm 7.1 *Local Regression At $X = x_0$*

1. Gather the fraction $s = k/n$ of training points whose x_i are closest to x_0.
2. Assign a weight $K_{i0} = K(x_i, x_0)$ to each point in this neighborhood, so that the point furthest from x_0 has weight zero, and the closest has the highest weight. All but these k nearest neighbors get weight zero.
3. Fit a *weighted least squares regression* of the y_i on the x_i using the aforementioned weights, by finding $\hat{\beta}_0$ and $\hat{\beta}_1$ that minimize

$$\sum_{i=1}^{n} K_{i0}(y_i - \beta_0 - \beta_1 x_i)^2. \tag{7.14}$$

4. The fitted value at x_0 is given by $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$.

Changing the span



Section 4

General Additive Models (GAMs)

Putting it all together

Yet another way to extend linear models

General Additive Model

Learning Y using $X_1, \cdots, X_p$

Old version:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

GAM:

function on 2nd variable

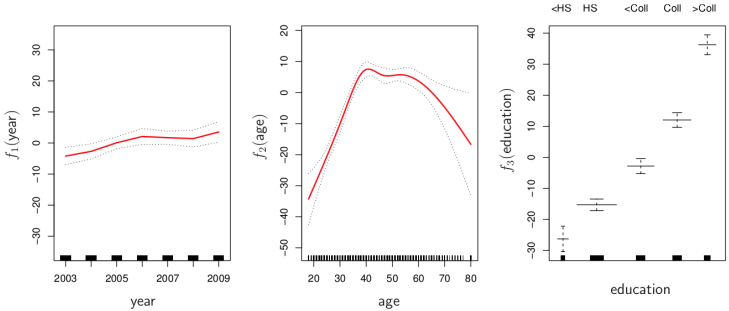
$$y_i = \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \varepsilon_i$$

$$= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_{\cancel{d}\,p}(x_{i\cancel{d}\,p}) + \varepsilon_i$$

additive

The trick

Use the previous tools (Ch 7.1-7.6) for extending linear models as the functions here



$$\texttt{wage} = \beta_0 + f_1(\texttt{year}) + f_2(\texttt{age}) + f_3(\texttt{education}) + \epsilon$$

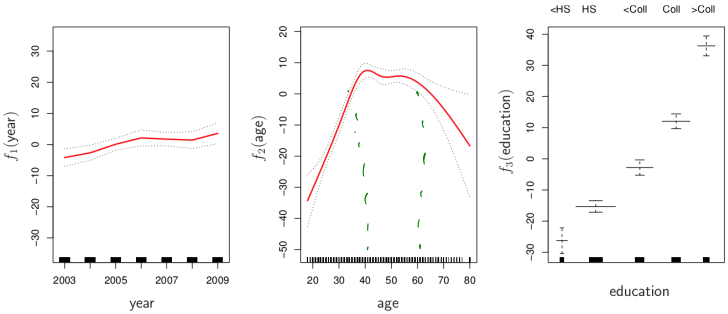
quantitative variables

natural splines

qualitative

dummy variables for this

Interpretation



holding year and age fixed more edu means higher income

$$\texttt{wage} = \beta_0 + f_1(\texttt{year}) + f_2(\texttt{age}) + f_3(\texttt{education}) + \epsilon$$

Holding age and edu. fixed wage increases slightly w/ year

holding year + edu fixed, highest income is for intermediate ages

GAMs for Classification

Learning qualitative Y using $X_1, \cdots, X_p$

Old version:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

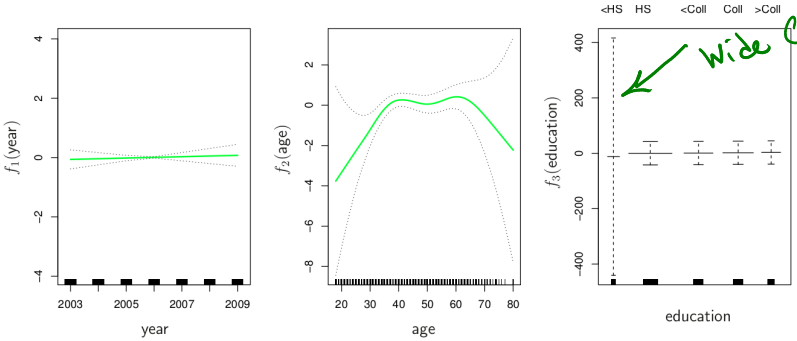
GAM:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p)$$

GAM Classification Example

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 \times \texttt{year} + f_2(\texttt{age}) + f_3(\texttt{education})$$

$$p(X) = \Pr(\texttt{wage} > 250 | \texttt{year}, \texttt{age}, \texttt{education})$$



Wide Confidence Interval

Reason:

No earners < HS

w/ > $250K

income

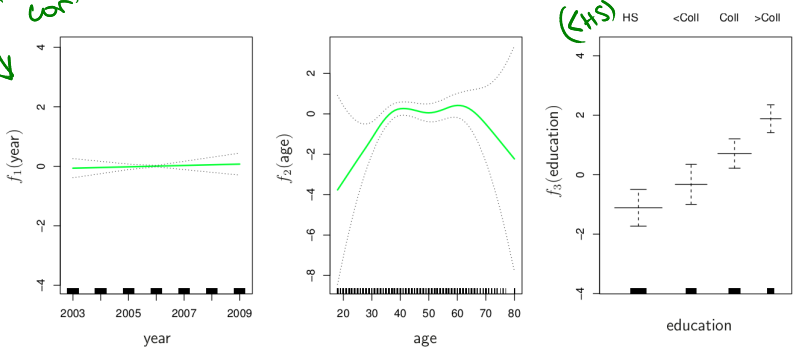
GAM Classification Example Part 2

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 \times \texttt{year} + f_2(\texttt{age}) + f_3(\texttt{education})$$

$$p(X) = \Pr(\texttt{wage} > 250 | \texttt{year}, \texttt{age}, \texttt{education})$$

not much contribution

Removed <HS from data



GAMs Pros

- Lots of options for functions
- Nonlinear terms for each x_j
  not manual checking on
  each variable
- Potentially better models
- Additivity => easy interpretation "hold every thing else fixed"

GAMs Cons/Limitations

- Restricted to be additive
- Could be interactions that are missed

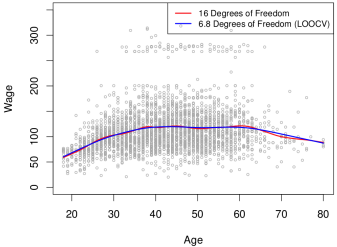
  ↳ add interaction terms $x_i \cdot x_j$

TL;DR

Smoothing Splines:

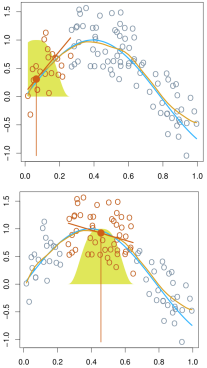
Find g that minimizes

$$\sum_{i=1}^{n}(y_i - g(x_i))^2 + \lambda \int g''(t)^2\, dt$$



Easy LOOCV trick

Local Regression:



Controlled by s = fraction of training points to use each time

GAMs:

$$\texttt{wage} = \beta_0 + f_1(\texttt{year}) + f_2(\texttt{age}) + f_3(\texttt{education}) + \epsilon$$

Use different function for each variable

Next time

Next time:

- Ch 7 lab

This week:

- Mon 3/14: 7.1-7.4
- Weds 3/16: 7.5-7.7
- Fri 3/18: Lab: 7.8
  - HW #7 Due Friday
  - Covers PCA/Ridge/Lasso Lab. Also content from today and Wednesday.